

综合练习 10: 编程

● 指南中与编程相关的内容:

3、单向链表掌握要求到会编程。

4、3 题编程 (1、基本类定义; 2、包含动态空间处理的类定义; 3、链表的理解和使用, 链表类的使用)

需要掌握的内容清单里:

4、 单向链表的结点和链表基本概念; 链表基本操作 (链表建立、链表结点遍历、添加、删除等), 会使用链表类, 能使用链表类的基本工作接口完成稍微复杂一些的工作。(比如使用链表类的当前结点数据访问 Data()、访问下一个结点 Next()、删除 Delete()和添加在表首 AddinFront()这四个基本接口完成链表的转置)

一、基本类定义: 掌握类定义的基本语法, class 的使用, 基本的构造析构的定义语法, 跟描述相关的数据成员和成员函数的定义语法, 跟描述相关的运算符重载的定义语法。

1、关注课本中出现过的相对完整的类定义

p253 例 9.4 Box 类

p256 例 9.5 Student 类;

p299 例 10.1 + p304 例 10.2 + p314 例 10.7 + p316 例 10.8 Complex 类;

p311 例 10.5 Time 类;

补充: 关注课本中出现过的相对完整的继承模型类定义(2 层继承或者 3 层继承, 最多 3 层继承)

p370 例 11.10 Student<-Graduate

p372 例 12.2 Teacher<-Professor(+Date)

p386 例 12.1 Point<-Circle<-Cylinder;

例如: 定义一个圆类 Circle, 包括:

(1) 私有数据成员: 圆心横坐标 x、纵坐标 y, 圆半径 radius(均为 double 类型)

(2) 一个构造函数: 能初始化数据成员 x、y、radius (默认初始值均为 10)

(3) 两个成员函数: perimeter(): 返回圆形周长; area():返回圆形面积

题目要求:

(1) 定义上述 Circle 类;

(2) 以 Circle 为基类, 定义圆球类 Ball (公有继承);

(3) 类 Ball 含成员函数 area(): 返回球体表面积; 成员函数 volume(): 返回球体体积。

(4) 编写主函数。在主函数中, 定义一个 Circle 对象, 该对象的 x、y、radius 值均由用户指定, 调用 perimeter()和 area()输出该对象的周长及面积; 再定义一个 Ball 对象, 该对象的 x、y、radius 去默认值, 调用 area()函数输出该 Ball 对象的表面积; volume()函数输出该 Ball 对象的体积。

(5) 提示: 球的表面积= $4\pi r^2$, 球的体积= $(4/3)\pi r^3$, r 为球半径

参考解答: (类似的继承模型很多, 要掌握根据描述定义基类, 基类数据成员访问控制没有特别说明的情况下为 `protected` (有说明按说明定义), 派生类的继承语法, 以及派生类内只专注派生类自己成员定义, 以及派生类使用基类继承成员完成自己的接口定义方法)

```
#include <iostream>
using namespace std;
const double PI = 3.14;
class Circle {
public:
    Circle(double xx = 10, double yy = 10, double r = 10) {
        x = xx; y = yy; radius = r; } //构造函数
    double area() { return radius * radius * PI; } //area()函数
    double perimeter() { return 2 * PI * radius; } //perimeter()函数
    double getRadius() { return radius; }
private:
    double x, y, radius; //数据成员定义
};
class Ball : public Circle { //继承
public:
    Ball(double xx = 10, double yy = 10, double r = 10):Circle(xx, yy,
r) {} //构造函数 1 分
    double area() { return 4* Circle::area(); } //area()
    double volume() { return area()*getRadius()/3; } //volume()
};
int main() {
    Circle c(1, 2, 3); //Circle
    cout << "Circle's perimeter is " << c.perimeter() << ", " << "Circle's
area is " << c.area() << endl; //函数调用
    Ball cy; //Cylinder 对象定义
    cout << "Ball's superficial area is " << cy.area() << endl;
    cout << "Ball's volume is " << cy.volume() << endl; //调用函数
    return 0;
}
```


2、请定义一个三角形类 Triangle, 包括:

- (1) 私有数据成员: 三条边长 (double 类型)
- (2) 一个构造函数: 能初始化这三条边长 (默认初始值均为 1)
- (3) 下列成员函数:

1) perimeter(): 返回三角形的周长;

2) isValid(): 如果三条边能构成一个合法三角形, 则返回 true, 否则返回 false.

定义该类, 并编写主函数. 在主函数中, 定义一个 Triangle 对象, 该对象的三条边长由用户输入, 然后调用 isValid 函数判断该对象是不是一个合法三角形. 如果不是合法三角形, 显示“不是一个合法三角形”; 如果是合法三角形, 显示周长.

参考解答: (这种类似的题可以出很多, 矩形, 正方形, 原型, 球等等,, 要掌握基本类定义语法, 能根据描述给出类的声明)

```
#include <iostream>
```

```
using namespace std;
```

//类定义方法 1: 可能类声明和函数实现内联, 掌握对应的语法

```
class Triangle {
    double a,b,c;
public:
    Triangle(double va=1,double vb=1,double vc=1):a(va),b(vb),c(vc){ }
    double perimeter(){ return a+b+c;}
    bool isValid(){
        if(a<=0 || b<=0 || c<=0) return false;
        return a+b>c && a+c>b && b+c>a;
    }
};
```

或者:

//类定义方法 2: 有时候会要求声明和定义分离, 也要掌握分离的语法

```
class Triangle {
    double a,b,c;
public:
    Triangle(double va=1,double vb=1,double vc=1);
    double perimeter();
    bool isValid();
};

Triangle::Triangle(double va,double vb,double vc):a(va),b(vb),c(vc){ }
```

```
double Triangle::perimeter(){ return a+b+c;}
```

```
bool Triangle::isValid(){  
    if(a<=0 || b<=0 || c<=0) return false;  
    return a+b>c && a+c>b && b+c>a;  
}
```

//按要求给出主函数

```
int main() {  
    double x,y,z;  
    cin >> x >> y >> z;  
    Triangle xobj(x,y,z);  
    if(xobj.isValid())  
        cout << "周长: " << t.perimeter()<< "\n";  
    else cout << "不是一个合法三角形\n";  
    return 0;  
}
```

3、有以下 `myInteger` 类声明描述的框架，它尝试定义一个自己的整型类，并能通过以下主函数测试。现在请你按照提示完善它，并给出相关的实现。

```
class myInteger{  
public:  
    myInteger(double vx=0);  
    myInteger operator+(const myInteger& right);  
    myInteger& operator++();  
    myInteger operator--();  
    //其他声明补充  
private:  
    int myint;  
};  
ostream& operator<<(ostream& out,const myInteger& obj);
```

```
int main(){  
    myInteger a(3.6),b(5),c;  
    c=a+b; cout<<c<<endl;  
    ++a;   cout<<a<<endl;  
    c=-a;  cout<<c<<endl;
```

```
b++;    cout<<b<<endl;
```

```
c=6+a; cout<<c<<endl;
```

```
if(a>b) cout<<">"<<endl;
```

```
else cout<<"<="<<endl;
```

```
return 0;
```

```
}
```

程序运行输出:

8

4

-4

6

10

<=

参考解答: (这种类似的题可以出很多, 复数类, 分数类, 时间类等等, 要掌握基本类定义语法, 课堂上介绍的几种运算符重载的方法, 能根据测试主程序给出类的声明)

(1) 补充类内声明:

根据 b++ 补充: myInteger operator++(int);

根据 a>b 补充: bool operator>(const myInteger& right);

根据 6+a 补充: operator int();

根据<< 补充: friend ostream& operator<<(ostream& out,const myInteger& obj);

(2) 实现:

//构造函数

```
myInteger::myInteger(double vx){
```

```
    myint=int(vx); //不做显示类型转换也可
```

```
}
```

//+运算重载

```
myInteger myInteger::operator+(const myInteger& right){
```

```
    myInteger result;
```

```
    *(result.myint)=*myint+*(right.myint);
```

```
    return result;
```

```
}
```

//++前置

```
myInteger& myInteger::operator++(){  
    ++(*myint);  
    return *this;  
}
```

//++后置

```
myInteger myInteger::operator++(int){  
    myInteger temp(*this);  
    ++(*myint);  
    return temp;  
}
```

//求相反数

```
myInteger myInteger::operator-(){  
    myInteger temp(*this);  
    *(temp.myint)=-(*myint);  
    return temp;  
}
```

//>重载

```
bool myInteger::operator>(const myInteger& right){  
    return *myint>*(right.myint);  
}
```

//类型转换重载

```
myInteger::operator int(){  
    return *myint;  
}
```

//输出流重载

```
ostream& operator<<(ostream& out,const myInteger& obj){  
    out<<*obj.myint;  
    return out;  
}
```


}

二、包含动态空间处理的类定义

注意：所有涉及动态空间的类首先要关注 4 个类的可缺省成员：基本构造函数/拷贝构造函数/赋值重载函数/析构函数

1、动态空间为单元空间模型，比如上面例题中的 `myInteger`，这种相对来说比较简单。

请自行补充完善 `myInteger` 的无参构造函数、拷贝构造、赋值重载和析构，参考解答如下：

```
(1) myInteger::myInteger(){
    myint=nullptr; //或者 myint=new int(0); 设置一个固定初始值也可以
}

(2) myInteger::myInteger(const myInteger& other){
    if(other.myint) myint=new int(*(other.myint));
    else myint=nullptr;
}

(3) myInteger& myInteger::operator=(const myInteger& other){
    if(this!=&other) *myint=*(other.myint);    //不是自赋值的处理
    return *this;
}

(4) myInteger::~~myInteger(){
    if(myint) delete myint;
}
```

2、动态空间为成组的空间模型，比如 `string`（字符串类），`vector`（向量类），`stack`（栈类），人员类（其中姓名分量的处理跟 `string` 类中的空间处理方式一样）

注意：一样需要关注无参构造/拷贝构造/赋值重载/析构；同时需要关注其他操作与空间相关的部分。这个方向做完实验 4 的同学应该不会有大问题，认真读题，理解好要你处理的空间模型即可。

三、链表的理解和使用（掌握链表的基本操作），链表类的使用（会使用链表类的成员函数实现稍微复杂的工作--比如使用链表类的当前结点数据访问 `Data()`、访问下一个结点 `Next()`、删除 `Delete()`和添加在表首 `AddinFront()`这四个基本接口完成链表的转置)

1、关注链表的初、中级基础操作（建表、添加结点、删除结点、链表查找、链表遍历、链表回收/删除整表）

关注链表的高级部分工作（链表转置、链表排序、链表一般合并/有序合并、链表排序等）

--除了理解基础实现代码以外，也要会使用链表类的接口完成这些相对复杂的工作。

这个题类根据指南关联内容给出两种角度可能的出题模型，请大家掌握基本链表编程方法以及使用类接口编程的方法。

2、有单向链表类 List、结点类 node 的部分定义以及主函数如下。注：链表类 List 中的结点为 node 类对象；链表类数据成员是指向链表第一个结点的指针表首。

要求：在类内实现(1)根据给定的 n 个元素的数组 array 创建链表的工作 Create();(2)链表的逆置工作，成员函数 ListReverse();(3)遍历链表结点依次输出结点值的工作 Display()

假设下列 List 类成员函数已定义完整，可以直接使用。

```
#include<iostream>
using namespace std;
//=====
class node {
public:
    node(int d = 0, Node* n = NULL) { data= d; next= n; }
    int data;
    node* next;
};//-----
class List {
    node* head; //链表表首
public:
    List() { head = NULL; }
    void Create(int *array, int n); //根据数组 array 的 n 个数据，创建链表
    void ListReverse(); //逆置链表
    void Display(); //遍历链表，依次输出结点值
};//-----
int main() {
    List aList; //创建空链表;
    node* p;
    int a[10] = { 1, 2, 3, 4, 5, 6, 7, 8, 9, 10 };
    aList.Create(a,10);
    aList.Display();
    aList.ListReverse();
    aList.Display();
    return 0;
```



```
//=====
```

程序输出结果为:

1, 2, 3, 4, 5, 6, 7, 8, 9, 10

10, 9, 8, 7, 6, 5, 4, 3, 2, 1

参考解答:

(1)void List::Create(int *array, int n){ //根据数组 array 的 n 个数据, 创建链表

```
    if(n>=1){
        node *tail;
        tail=head=new node(array[0]);
        for(int i=1;i<n;i++){
            tail->next=new node(array[i]);
            tail=tail->next;
        }
    }
}
```

(2)void List::ListReverse(){//逆置链表

```
    node *newhead=NULL;
    while(head){
        node* p=head;
        head=head->next;
        p->next=newhead;
        newhead=p;
    }
    head=newhead;
}
```

(3)void List::Display(){//遍历链表, 依次输出结点值

```
    node *p=head;
    if(p){ cout<<p->data; p=p->next; }
    while(p){
        cout<<','<<p->data;
        p=p->next;
    }
}
```

```

        cout<<endl;
    }

```

3、有单向链表类 List、结点类 node 的部分定义以及主函数如下。注：链表类 List 中的结点为 node 类对象；链表类数据成员是指向链表第一个结点的指针表首和链表当前工作指针。要求：在类外实现(1)根据给定的 n 个元素的数组 array 创建链表的工作 Create();(2)链表的逆置工作，成员函数 ListReverse();(3)遍历链表结点依次输出结点值的工作 Display() 假设下列 List 类成员函数已定义完整，可以直接使用。

```

#include<iostream>
using namespace std;
//=====
class node {
public:
    node(int d = 0, Node* n = NULL) { data= d; next= n; }
    friend List;
private:
    int data;
    node* next;
};//-----
class List {
public:
    node* head; //链表表首
    node* cur; //当前结点指针

    List() { cur= head = NULL; }
    int Data(){ //返回 cur 的 data 值
        if(cur) return cur->data;
        return -1;
    }
    node* Next(){//返回 cur 的 next 值
        if(cur) return cur->next;
        return NULL;
    }
    void Delete(){//删除当前结点 cur,删除后设置 cur 为被删除结点的后继；若删除头结点，则 cur 为新表首

```

```

    if(cur){
        node *p=head;
        if(cur==head){ //cur 为 表首点
            cur=head=head->next;
            delete p;
        }
        else{ //cur 为其他
            while(p&& p->next!=cur) p=p->next;
            p->next=cur->next; //旁路构建
            delete cur;
            cur=p->next; //cur 为后继点
        }
    }
}

void AddinFront(int val){//添加值为 val 的新结点到表首，并更新表首和 cur 为该新结点。
    head=new node(val,head);
    cur=head;
}

};//-----

void Create(List& xlist,int *array,int n());//根据数组 array 的 n 个数据，创建链表
void ListReverse(List& xlist);//逆置链表
void Display(List& xlist);//遍历链表，依次输出结点值

```

```

int main() {
    List aList;//创建空链表;
    int a[10]={ 1, 2, 3, 4, 5, 6, 7, 8, 9, 10 };
    Create(aList,a,10);
    Display(aList);
    ListReverse(aList);
    Display(aList);
    return 0;
};//=====

```

程序输出结果为:

```

1, 2, 3, 4, 5, 6, 7, 8, 9, 10
10, 9, 8, 7, 6, 5, 4, 3, 2, 1

```


参考解答:

(1)void Create(List& xlist,int *array,int n){//根据数组 array 的 n 个数据, 创建链表

```
    for(int i=9;i>=0;i--){
        xlist.AddinFront(array[i]);
    }
```

(2)void ListReverse(List& xlist){//逆置链表

```
    int *mem,c=0;
    xlist.cur=xlist.head; //先按结点个数数一遍
    while(xlist.cur) {
        c++;
        xlist.cur=xlist.Next();
    }
    mem=new int[c]; //按数到的个数分配空间
    xlist.cur=xlist.head;
    c=0;
    while(xlist.cur) { //遍历链表存放值, 并把链内结点删光
        mem[c++]=xlist.Data();
        xlist.Delete();
    }
    for(int i=0;i<c;i++) //根据存放的值逆置构建
        xlist.AddinFront(mem[i]);

    delete []mem;    //释放临时空间
}
```

(3)void Display(List& xlist){//遍历链表, 依次输出结点值

```
    xlist.cur=xlist.head;
    if(xlist.cur) cout<<xlist.Data();
    xlist.cur=xlist.Next();
    while(xlist.cur){
        cout<<','<<xlist.Data();
        xlist.cur=xlist.Next();
    }
    cout<<endl;
```

